



Run of show and full presentation script

Loom: a decentralised, verifying package manager for the AUR

This is the whole presentation from the first word to the last question, in order: who speaks, what is on the screen, what the others are doing, what to press during the live demo, and the exact words. The four individual speaker PDFs contain the same text split by presenter, plus the Q&A preparation.

1. Roles

Slot	Speaks	Controls	When not speaking
Presenter 1	Slides 1–7: introduction, problem, literature review, research gap	Clicker for slides 1–7	Timekeeper from slide 8 to the end
Presenter 2	Slides 8–11: architecture, flows, cryptography, Warp	Clicker for slides 8–11	Timekeeper during Part 1; notes the jury's questions during the demo
Presenter 3	Slide 12 (Heddle and Weave), then live demo steps 1–5	Clicker for slide 12; laptop keyboard for steps 1–5	Before slide 12: at the laptop, keeping <code>present.sh</code> ready at step 1
Presenter 4	Live demo steps 6–9, slides 13–16, leads the Q&A	Keyboard for steps 6–9; clicker for slides 13–16	Watches the console during part A, ready to point at it

2. Stage and screen

- One laptop, **mirrored** to the projector, so what the presenter sees is what the jury sees. The demo needs no internet: every service runs on the laptop.
- **Two workspaces** (virtual desktops): workspace 1 has the slides in full-screen slideshow mode; workspace 2 has the terminal on the left half and the browser (<http://127.0.0.1:7790>) on the right half. Practise switching with Alt+Tab or your desktop's workspace shortcut.
- The terminal runs **demo/present.sh**, started before the jury arrives and left waiting at step 1. The browser has a second tab with **docs/demo-report.html** as the backup.
- Use a clicker for the slides so each speaker can stand, not sit at the laptop. Only the demo needs someone at the keyboard.

3. Countdown

When	Who	What
Day before	Everyone	Full rehearsal on the exact laptop, with the projector if possible. Time it: the talk should run about 33 minutes. Record one complete run of <code>demo/run.sh</code> as a last-resort video.
T – 30 min	Presenter 3	<code>cd loom · cargo build · python3 demo/fixtures/genfix.py · demo/run.sh</code> (dry run, must end with 'demo complete'). Check that it prints 'sandbox tier on this host: full'.
T – 15 min	Presenter 3	Start <code>demo/present.sh</code> ; all pre-flight checks show ; leave it at the step 1 banner. Arrange workspace 2 (terminal left, browser right, backup tab open). Terminal font ≥ 16 pt.
T – 5 min	Presenter 1	Slideshow open at slide 1 on workspace 1; clicker tested; phones silent; water.



4. Timeline at a glance

Target length about 33 minutes before questions. Each handover is marked in the full script below. If you are running late, drop the demo finale (step 9) and show docs/demo-report.html for 20 seconds instead: that saves about 70 seconds.

Clock	Who	Segment	On screen	Everyone else
0:00	Presenter 1	Slide 1 · Title	slides	P2 keeps time (cards at 2 min and 30 s before each handover) · P3 sits at the laptop, present.sh waiting at step 1 · P4 stands ready
0:30	Presenter 1	Slide 2 · Abstract and index terms	slides	"
1:30	Presenter 1	Slide 3 · Installing from the AUR means running a stranger's script	slides	"
2:45	Presenter 1	Slide 4 · Problem, objectives and the five defences	slides	"
4:00	Presenter 1	Slide 5 · Literature review (1 of 2)	slides	"
5:00	Presenter 1	Slide 6 · Literature review (2 of 2)	slides	"
6:00	Presenter 1	Slide 7 · Research gap	slides	"
7:00	Presenter 2	Slide 8 · System architecture	slides	P1 takes over timekeeping for the rest of the talk · P3 at the laptop · P4 stands ready
9:00	Presenter 2	Slide 9 · Install flow: decide before anything runs	slides	"
11:00	Presenter 2	Slide 10 · Attestation and logging flow (sequence)	slides	"
12:45	Presenter 2	Slide 11 · Cryptography and the Warp transparency log	slides	"
14:45	Presenter 3	Slide 12 · Heddle sandbox and Weave policy engine	slides	P1 keeps time · P2 notes the jury's questions and reactions for Q&A; · P4 watches the console, ready to take the keyboard
18:15	Presenter 3	Switch to workspace 2 (terminal + browser)	demo workspace	"
18:35	Presenter 3	Demo step 1 · A healthy package: verify, don't build	terminal + console	"
19:35	Presenter 3	Demo step 2 · Temporal quarantine and the advisory fast-path	terminal + console	"
20:40	Presenter 3	Demo step 3 · The orphan-adoption attack	terminal + console	"
21:55	Presenter 3	Demo step 4 · Build-time network injection (npm install inside a PKGBUILD)	terminal + console	"
23:00	Presenter 3	Demo step 5 · The sandbox contrast: same build, confined vs unconfined	terminal + console	"
24:15	Presenter 4	Demo step 6 · Force-push plus a planted pacman hook (defence in depth)	terminal + console	P1 keeps time · P2 notes questions · P3 steps back from the keyboard, ready to answer sandbox/policy questions



Clock	Who	Segment	On screen	Everyone else
25:30	Presenter 4	Demo step 7 · A Python .pth startup hook (the LiteLLM vector)	terminal + console	"
26:15	Presenter 4	Demo step 8 · A split-view attack on the log	terminal + console	"
27:30	Presenter 4	Demo step 9 · Finale — all nine scenarios with the live console (optional, about 50–70 seconds)	terminal + console	"
29:00	Presenter 4	Press Enter ('Demo complete'); switch back to the slideshow at slide 13	slides	"
29:20	Presenter 4	Slide 13 · Results: every checked acceptance criterion passes	slides	"
30:50	Presenter 4	Slide 14 · Live demonstration recap	slides	"
31:35	Presenter 4	Slide 15 · Conclusion	slides	"
33:05	Presenter 4	Slide 16 · References	slides	"
33:05	Presenter 4	Questions and answers (references slide stays up)	slide 16	everyone answers in their own area



5. The full script, start to finish

Grey lines are actions. **Say** paragraphs are what the speaker says. Amber boxes are handovers.

[0:00] Presenter 1 · Slide 1 — Title

Action: Presenter 1 walks to the front with the clicker; slide 1 is already on screen.

On screen: the woven pattern on the right is our logo motif; the name Loom and its parts (Warp, Weft, Heddle, Thread, Shuttle) come from weaving.

Good morning. We are Group 14: Aiman Haque, Zoya Mulani, Aariz Sheikh and Yusuf Aslam. Our major project is **Loom: a decentralised, verifying package manager for the Arch User Repository**, guided by [guide's name].

In one sentence: Loom lets you install community software from the AUR safely, without needing any cooperation from the AUR or from the people who publish packages there. It does that by containing every build, by demanding independent, publicly logged evidence before anything is installed, and by noticing when the people behind a package change.

[0:30] Presenter 1 · Slide 2 — Abstract and index terms

On screen: point at the four numbers on the right as you mention them.

The abstract follows the IEEE format. The key idea is in the second sentence: community repositories run unreviewed build scripts with your full privileges, and they install whatever a maintainer publishes.

Loom is **not a malware scanner**. It combines five independent mechanisms: a least-privilege build sandbox called Heddle; Thread and Weave, which only accept an artifact when several independent rebuilders reproduce it bit-for-bit; a temporal quarantine for brand-new versions; continuity checks that notice a change of maintainer or a rewritten history; and Warp, a transparency log cosigned by independent witnesses.

On the right are four results from our own evaluation, which Presenter 4 will show in detail: nine of nine malicious cases blocked, zero of seven benign packages blocked, two hundred of two hundred split-view attacks detected, and sixty microseconds to verify an attestation signature.

[1:30] Presenter 1 · Slide 3 — Installing from the AUR means running a stranger's script

On screen: trace the four steps top to bottom; steps 3 and 4 are red because that is where the damage happens.

Let me show why this problem matters. On the left is what happens when you type **yay -S** some package. The helper downloads a PKGBUILD from the AUR's git server. makepkg then **runs that PKGBUILD, a shell script, as your own user**. Inside its build function it can read your SSH keys, your cloud credentials and your browser data, and it can open network connections. Only after that does pacman install the result as root, running the package's install hooks as root too.

Nobody reviews uploads to the AUR, nothing is signed, and when a maintainer abandons a package, **anyone can adopt it** and push a new version to every existing user. Miller et al. [6] showed that abandonment is common and that users have no plan for it. The XZ Utils backdoor in 2024 [5] is the textbook case: a patient takeover of maintainership followed by a payload hidden in the build process.



On the right, the scale, all from peer-reviewed work since 2023: 107 distinct attack vectors on open-source supply chains [1]; 24,356 malicious packages in the largest in-the-wild dataset [4]; three quarters of malicious PyPI packages reached users **through source installation** [3], which is exactly how the AUR works; and fewer than one percent of packages sandbox themselves [15]. So containment has to be enforced by the package manager. It cannot be left to package authors.

[2:45] Presenter 1 · Slide 4 — Problem, objectives and the five defences

On screen: walk the five cards left to right; each has an objective at the top and the attack it counters at the bottom.

Our problem statement is in the dark box: install AUR packages only when there is independent, publicly verifiable evidence that the artifact matches its source; contain every build under least privilege; and detect changes of publishing authority; all with **no cooperation** from the AUR, its maintainers or upstream. That last constraint is what makes the problem hard.

We turned that into five objectives, and each is met by one subsystem. One: contain every build, handled by **Heddle**. Two: verify, don't trust, handled by **Thread and Weave**. Three: slow down fast attacks, handled by **Quarantine**. Four: notice who changed, handled by **Continuity**. Five: make the evidence public, handled by **Warp**. A sixth objective runs through all of them: every decision must explain which rule fired and how to proceed.

The mechanisms are deliberately independent, so an attacker who beats one still faces four. The footer states what we do not claim, for example that if every rebuilder faithfully reproduces malicious source code, reproducibility alone cannot tell. We state this openly because it is exactly why we have five mechanisms and not one.

[4:00] Presenter 1 · Slide 5 — Literature review (1 of 2)

On screen: don't read the table; pick the highlighted phrases.

We reviewed sixteen papers, all from 2023 onwards and from venues such as IEEE S&P, ACM CCS, USENIX Security, NDSS, ASE, FSE, ICSE, MSR and DSN. Each row gives the contribution and the gap Loom fills.

The first four characterise the threat. Ladisa et al. [1] built the reference taxonomy of supply-chain attacks. Ladisa [2], Guo [3] and Zhou [4] study malicious packages; their detectors are useful but **probabilistic**, so an attacker can adapt, which is why Loom contains and verifies instead of classifying.

Przymus and Durieux [5] reconstruct the XZ Utils attack, and Miller et al. [6] study abandonment. Together they tell us the attack surface is changes of authority plus build-time payloads, which our continuity checks and sandbox target. Schorlemmer [7] and Merrill [8] cover signing: signing tells you **who** published, not whether the artifact matches its source, and it needs registry cooperation that the AUR does not offer.



[5:00] Presenter 1 · Slide 6 — Literature review (2 of 2)

The second half covers the building blocks we use. Abu Ishgair et al. [9] argue for layered defences evaluated by the attacks they actually stop, which is exactly how we evaluate Loom, with an ablation study. Tamanna et al. [10] show that provenance frameworks like SLSA stall because they need the producer's buy-in; Loom produces its own provenance through independent rebuilders.

Malka, Zacchiroli and Zimmermann [11, 12] show that independent rebuilding is feasible at scale: over 700,000 Nix packages rebuilt. But nobody turns reproducibility into an install-time decision, which is what our Weave engine does.

Parakeet [13] and OPTIKS [14] show that transparency logs work in production without any blockchain; Warp applies that idea to build attestations. Finally, Alhindi and Hallett [15] found that sandboxing is rarely adopted by developers, and Brandt et al. [16] validate the Landlock-plus-seccomp combination that Heddle relies on.

[6:00] Presenter 1 · Slide 7 — Research gap

On screen: run your finger down the green Loom column, then across the bottom row.

This table is the conclusion of our review. Each column is a class of existing solution and each row a property. yay and paru add convenience, not verification. A clean chroot build helps a little with isolation. Malware detectors are probabilistic. Signing needs registry cooperation. Reproducible-builds research measures reproducibility but never makes an install decision.

Only the last column has every property, and it is the only one that also **works with zero registry cooperation**. That combination, verified at install time on the client, is our contribution.

Handover · Presenter 1 says:

“So that is the problem and the gap. [Presenter 2] will now show you how Loom is built: its architecture, and what happens step by step when you install a package.”

[7:00] Presenter 2 · Slide 8 — System architecture

On screen: trace AUR → rebuilders → Warp → witnesses, then the long indigo arrow back into the client.

Here is the whole system. The user talks only to the **loom client**, the dashed box, which is our trusted computing base. Inside it: **Weave**, the policy engine that makes decisions; **Heddle**, the build sandbox; the **Warp client**, which verifies the log; **Shuttle**, which fetches artifacts by hash; and the AUR backend, the only part that knows anything about the AUR.

Outside are independent parties. The **AUR** is read-only to us: we read recipes, and we never execute a recipe to learn its metadata; we parse the .SRCINFO file instead. Three **Thread rebuilders**, run by different organisations with different toolchains, rebuild every package and sign what they built. Those signed attestations go into the **Warp log**, and three **witnesses** cosign the log's checkpoints. **Shuttle peers** serve the built artifacts by content address.

At the bottom, pacman performs the final install and the Linux kernel provides the sandbox primitives. The key point is the note on the right: every arrow into the client carries data that is verified by a hash or a signature before it is used. If any verification fails, Loom refuses. It never falls back to trusting the data.



[9:00] Presenter 2 · Slide 9 — Install flow: decide before anything runs

On screen: follow the numbers: top row left to right, the diamond, then the bottom row right to left.

This is what happens when you type **loom install**. Step one resolves the package and its dependencies, dependencies first. Step two fetches the recipe and parses .SRCINFO, and no package code runs. Step three gathers evidence: it verifies the log, collects the attestations, and checks continuity, quarantine and security advisories.

Step four is the Weave decision. If anything fails, Loom refuses right there and prints the rule that fired, the evidence and the exact command to proceed if the user accepts the risk. **At that point no package code has executed.**

If it is allowed, step five obtains the artifact. Loom prefers the **attested artifact**, fetched from peers by its SHA-256 hash. So when k independent rebuilders agree, the PKGBUILD never runs on the user's machine at all. Only if no peer can serve it does Loom build locally, inside Heddle.

Then Loom inspects what the package would install, strips its install script so pacman can never run it as root, and re-evaluates, because the files themselves can still cause a block, for example a pacman hook. Finally it installs, runs any install script inside the sandbox, and records a baseline for future continuity checks.

[11:00] Presenter 2 · Slide 10 — Attestation and logging flow (sequence)

On screen: move down the numbered arrows 1 to 10.

This sequence diagram shows where the evidence comes from; time runs downward. A rebuilder fetches the recipe with its pinned sources and **builds it twice** with deliberately different time zones, locales, umasks and build paths. That is the reprotest technique from the Reproducible Builds project. If both SHA-256 digests match, the build is reproducible.

The rebuilder then signs an in-toto statement, wrapped in a DSSE envelope, with its own Ed25519 key. The statement binds the artifact hash to the source commit and to the rebuilder's identity and toolchain. It submits this to Warp.

Warp checks that the signer is an admitted rebuilder, appends the record as a new leaf, signs a new checkpoint and sends it to each witness together with a **consistency proof**. A witness verifies that proof against the last checkpoint it cosigned and only then returns a cosignature.

Finally the client fetches the checkpoint, requires at least two of the three cosignatures, checks consistency with the head it saved last time, and verifies inclusion of every attestation it uses.

[12:45] Presenter 2 · Slide 11 — Cryptography and the Warp transparency log

On screen: point at the code box when you say 0x00 and 0x01; trace $d2 \rightarrow h1.1 \rightarrow h2.0 \rightarrow \text{root}$, then the three amber boxes.

We invent no cryptography; that is a hard constraint in our requirements. Everything uses two audited Rust libraries: ed25519-dalek and sha2. **SHA-256** gives every artifact and record a content address. **Ed25519** signs attestations, checkpoints and cosignatures, and we verify with `verify_strict`, which rejects malleable and small-order signatures.

Two subtler choices matter. **DSSE's pre-authentication encoding** signs the payload type and the body, each prefixed with its length, so nothing can be spliced or reinterpreted; and because the



in-toto *predicate* type is inside the signed body, a signature on an attestation can never be reused as a signature on a revocation. And **canonical JSON**, with sorted keys and no floating-point numbers, means the same record always produces the same bytes, so anyone can re-verify a signature.

On the right is the Merkle tree. Leaves are hashed with a 0x00 prefix and interior nodes with 0x01, as in RFC 6962; that domain separation stops an attacker from passing off an interior node as a leaf. To prove that record d2 is in the log, the server sends only the three amber sibling hashes: log base two of eight. With a million records that is only twenty hashes.

Why witnesses? A dishonest log could show the victim one history and everyone else another: a **split view**. A witness only cosigns after checking consistency with what it cosigned before, so an honest witness vouches for one linear history. With two of three witnesses required, any two quorums share at least one witness, so a split view is **prevented** while at most one witness colludes, and detected beyond that.

Handover • Presenter 2 says:

“So the evidence is signed, logged and witnessed. What stops a malicious build in the first place, and how does Loom turn all this evidence into a decision? [Presenter 3] will explain Heddle and Weave, and then show you all of this running live.”

[14:45] Presenter 3 • Slide 12 — Heddle sandbox and Weave policy engine

On screen: left half first, from the outer box inwards; then the red box; then the nine rule chips and the three-node graph.

On the left is **Heddle**, our build sandbox, made of three nested layers. The outer layer is Linux **namespaces**: the build gets its own user, mount, network, process, IPC, hostname and cgroup namespaces. We pivot_root into a fresh, empty temporary filesystem that contains only the toolchain, read-only, and the build directory. Your home directory does not exist inside, and the network namespace is empty.

The middle layer is **Landlock**, a Linux security module that any unprivileged process can apply to itself. It is a kernel allow-list for files, TCP ports and inter-process communication. The inner layer is a **seccomp-BPF** filter that refuses dangerous system calls such as mount, ptrace, bpf and io_uring, and hands socket and file-open calls to a supervisor that **explains** each denial in the terms of our requirements.

Tiers: if the host disallows user namespaces we drop to the reduced tier, Landlock plus seccomp, and say so. If even Landlock is missing, we **refuse to build**. There is never an unconfined fallback.

The red box is a change we made while hardening the prototype. A malicious script can wrap its payload in `|| true` so that the build still succeeds after the sandbox denies it. We now treat any attempt to read credentials, or to use the network, as hostile: rebuilders refuse to attest such a build, and Weave blocks it, even when it exits with success.

On the right is **Weave**. It is a pure function from the policy and the evidence to a decision. It evaluates all nine rules and reports every violation, each with evidence and a remediation. Continuity runs first, because a change of maintainer can tighten the rules that follow.

The most interesting algorithm is **independence**. Two attestations are correlated if they share a rebuilder, an organisation or a toolchain; the toolchain ID is a SHA-256 hash of the build image and its



component versions. The number of independent attestations is the **maximum independent set** of that correlation graph. In this example, thread-b and thread-c used the same toolchain, so three agreeing attestations count as two. That still meets the default k of two.

Action: at the end of this slide, Presenter 3 switches to workspace 2 (terminal + browser), where present.sh is waiting at step 1.

[18:35] Presenter 3 · Demo step 1 — A healthy package: verify, don't build

Say first: “Everything you are about to see is real: a mock AUR, the Warp log, three witnesses and three rebuilders, running as separate processes on this laptop. The ‘malicious’ packages are harmless probes that only read a planted decoy file and ping a local test server; nothing leaves the machine.”

*Action: press **Enter** (present.sh step 1). Wait about 7 seconds until the screen says Step 1 is live.*

*Point at: Three rebuilders (thread-a, -b, -c) each print reproducible with the same short hash for libweft and hello-loom.; **loom log**: tree size, the root hash, cosigned by: witness-1, witness-2, witness-3.; **loom install hello-loom**: libweft is installed first (a dependency), both **ALLOWED**, artifact ... from peer http://127.0.0.1:773x, and post_install() ran under Heddle (full; exit 0; 0 denial(s)).*

“Three independent rebuilders reproduced these packages bit-for-bit and logged it. The log is cosigned by all three witnesses. Loom installed hello-loom by fetching the attested artifact from a peer by its hash, so **no PKGBUILD code ran on this machine**. Even the package's own install script ran inside the sandbox.”

*Action: press **Enter** to stop the step.*

[19:35] Presenter 3 · Demo step 2 — Temporal quarantine and the advisory fast-path

*Action: press **Enter** (present.sh step 2). Wait about 7 seconds until the screen says Step 2 is live.*

*Point at: **fastmover 0.9-1 — BLOCKED (pre-build)**, [FAIL] temporal quarantine, published 1h ago; quarantine is 3d — 2d 22h remaining, and the exact override command.; Then an advisory feed is loaded (CVE-2026-31337), and **tlspoke 2.0.1-1 — ALLOWED**, although it is just as new.*

“fastmover was published an hour ago, so it is held for 72 hours even though all three rebuilders reproduced it. Worms like Shai-Hulud spread within hours and are usually caught within days, so a short delay removes most of their reach. tlspoke is just as new, but the Arch security advisory feed says it fixes a vulnerability, so the quarantine is waived. We don't want to delay security fixes.”

*Action: press **Enter** to stop the step.*

[20:40] Presenter 3 · Demo step 3 — The orphan-adoption attack

*Action: press **Enter** (present.sh step 3). Wait about 7 seconds until the screen says Step 3 is live.*

*Point at: orphan-tool 1.0 by **bob** installs normally.; published orphan-tool 1.1-1 by mallory; then all three rebuilders print **buildfailed ... refused: build attempted openat /root/.curlrc [FR-3.2], socket ... AF_INET [FR-3.4].; orphan-tool 1.1-1 — BLOCKED (pre-build)**: [FAIL] publishing-authority continuity, maintainer changed: bob → mallory, with dates and the source of the evidence.*

“This models the 2026 AUR orphan-adoption waves. Bob abandoned the package; mallory adopted it and shipped 1.1. Loom remembers who published what you installed, notices the change of maintainer, and **blocks before building anything**.

Independently, look at the rebuilders: 1.1's build tried to read a file in root's home and open an internet socket. Heddle denied both, and the rebuilders refused to vouch for it. That is two separate



mechanisms catching the same attack.”

Action: press **Enter** to stop the step.

[21:55] Presenter 3 · Demo step 4 — Build-time network injection (npm install inside a PKGBUILD)

Action: press **Enter** (present.sh step 4). Wait about 7 seconds until the screen says Step 4 is live.

Point at: npm-helper 2.4 installs. carol's account is compromised and 2.5 is published.; All three rebuilders: **buildfailed ... refused: build attempted; loom verify npm-helper --build** → **BLOCKED (pre-install)**, [FAIL] build confinement: build tried to reach home/credential files and the network — contained by Heddle, artifact rejected (exit 0; 2 hostile access attempt(s)), followed by the list of denials.; **loom-testbed sink** → (sink empty — no build reached the network).

“This is the Atomic Arch pattern: a package whose build quietly runs npm install to fetch a payload. Notice **exit 0**: the script swallowed its own errors so the build looks successful. Loom does not care. The build tried to use the network and read credentials, so it is hostile. And the test server, our sink, received nothing.”

Action: press **Enter** to stop the step.

[23:00] Presenter 3 · Demo step 5 — The sandbox contrast: same build, confined vs unconfined

Action: press **Enter** (present.sh step 5). Wait about 7 seconds until the screen says Step 5 is live.

Point at: A decoy canary is planted in \$HOME.; (a) confined: **BLOCKED** with the two denials.; (b) UNCONFINED (demo backend, clearly marked): **ALLOWED WITH WARNINGS** — DEMO BACKEND: build ran WITHOUT kernel confinement.; **network sink hits: confined=0 unconfined=1** and HIT ... marker=loom-probe&pkg=npm-helper&stage=npm-install.

“This is our headline empirical result. The **same** malicious build, run twice. Confined by Heddle, zero hits: it could not read the home directory or reach the network. Unconfined, the way a normal AUR helper runs it, it phoned home. We don't just claim the sandbox works; we show it denying the payload.”

Action: press **Enter** to stop the step.

Handover · Presenter 3 says:

“That was containment and quarantine. [Presenter 4] will now show the attacks that try to hide: a rewritten history with a planted hook, a Python startup hook, and an attack on the log itself.”
(Press Enter to stop step 5. The screen shows HANDOVER and step 6 waits for Presenter 4.)

[24:15] Presenter 4 · Demo step 6 — Force-push plus a planted pacman hook (defence in depth)

Action: press **Enter** (present.sh step 6). Wait about 7 seconds until the screen says Step 6 is live.

Point at: forcepush-lib 3.1 installs. Then published forcepush-lib 3.1-1-rewrite by frank (history rewritten).; **BLOCKED (pre-build)**: [FAIL] continuity, recipe history rewritten (force-push): commit ... does not descend from it.; The user overrides the continuity alarm: **[OVRD]** continuity ... accepted by override #1, and still **BLOCKED (pre-install)**: [FAIL] placement, /usr/share/libalpm/hooks/zz-fp.hook — pacman hook: runs as root on every future package transaction.

“This models the TeamPCP incident, where more than a hundred tags were force-pushed. Loom proves the new commit does not descend from the one you installed, using git's ancestry check, so the history was rewritten. Blocked.



Now the interesting part: suppose the user reviews it and overrides that alarm. The package still carries a pacman hook, a file that would run as root on every future install. The **placement policy**, which looks at what a package installs, blocks it anyway. That is defence in depth, live.”

Action: press **Enter** to stop the step.

[25:30] Presenter 4 · Demo step 7 — A Python .pth startup hook (the LiteLLM vector)

Action: press **Enter** (present.sh step 7). Wait about 7 seconds until the screen says Step 7 is live.

Point at: **pth-inject 1.0-1 — BLOCKED (pre-install)**: [FAIL] installed files do not create persistence or privilege, .../site-packages/loomdemo.pth — .pth file with import lines: executes on every Python start (LiteLLM-style).

“A .pth file in site-packages runs on every Python start: a perfect persistence hook that no build sandbox can see, because nothing malicious happens at build time. That is how the LiteLLM compromise worked. Loom inspects the artifact's files before installing and blocks it.”

Action: press **Enter** to stop the step.

[26:15] Presenter 4 · Demo step 8 — A split-view attack on the log

Action: press **Enter** (present.sh step 8). Wait about 7 seconds until the screen says Step 8 is live.

Point at: Warp restarts in split-view mode with witness-3 corrupt. The victim syncs: size 6, cosigned by all three.; The operator forks the victim's view; thread-a and thread-b attest a new hello-loom commit to the public log; the compromised thread-c's **backdoored** attestation goes only into the fork.; **victim re-syncs**: transparency log verification failed: checkpoint (size 7) carries 1 valid witness cosignature(s) [witness-3]; policy requires 2 of 3.; An uninvolved client still sees the honest history (size 8, all three witnesses).

“Now the attacker is the log operator itself, colluding with one compromised rebuilder and one corrupt witness. It shows the victim a private history containing a backdoored attestation. But the honest witnesses have already cosigned the public history, and a fork does not extend it, so they refuse. The victim's client sees only one cosignature where it needs two, and **rejects the fork**. Everyone else is unaffected.”

Action: press **Enter** to stop the step.

[27:30] Presenter 4 · Demo step 9 — Finale — all nine scenarios with the live console (optional, about 50–70 seconds)

Action: press **Enter** (present.sh step 9). Wait about 7 seconds until the screen says Step 9 is live.

Point at: All nine scenarios stream past in the terminal. Switch to the browser tab <http://127.0.0.1:7790>.; The console: service health (witness-3 turns red as corrupt in scenario 8), the log drawn as coloured threads, and the defence matrix filling in, with red cells spread across every column.

“This is our deployment console, reading the same services live. Watch the defence matrix: each red cell is a mechanism stopping a package, and the red is spread across every column. No single defence does all the work. At the end the run saves this page as a report file, so the result can be inspected afterwards.”

Action: press **Enter** to stop the step. The screen shows Demo complete; switch back to the slideshow at slide 13.



[29:20] Presenter 4 · Slide 13 — Results: every checked acceptance criterion passes

On screen: table, then chart, then the four tiles.

Our specification defines six experiments. **E1**, attack replay: all nine malicious cases in our labelled incident corpus are blocked, and blocked **before any payload could run**. **E2**, false positives: none of the seven benign packages is blocked, even though they deliberately stress each mechanism, such as a fresh security release or a reviewed change of maintainer.

E3, build compatibility, is a measurement rather than a pass or fail gate; the live demo measures it. **E4** is the chart: the ablation study. Detections are spread across five mechanisms, and the best single one catches only three of nine, so no single bypass defeats Loom.

E5: a full policy decision takes 8.3 microseconds and verifying an Ed25519 attestation 60 microseconds in a release build, thousands of times under the targets, so installation time is dominated by the network. **E6**: 200 of 200 split views detected. And across the bottom, the sandbox contrast you saw live: zero hits confined, one unconfined.

[30:50] Presenter 4 · Slide 14 — Live demonstration recap

This table recaps what you just saw live, straight from the run's event journal. Rows are the nine scenarios and columns the mechanisms. Red cells show where a mechanism stopped a package. They appear in every column, so the ablation result also holds on the live system. Scenario 8's only red cell is the witnesses column: the fork was rejected.

[31:35] Presenter 4 · Slide 15 — Conclusion

To conclude: Loom turns installing community software into a verifiable decision instead of an act of faith. It **contains** every build, **verifies** artifacts through independent rebuilders and a witnessed log, **detects** changes of authority, fresh releases and persistence hooks, and **explains** every decision.

We are candid about the limitations. Rebuilder independence is simulated on one machine; a real deployment needs independent operators. Attestation proves that an artifact matches its source, not that the source is benign. And admission to the log is permissioned. Future work follows from these: a real federation of rebuilders and witnesses; npm, PyPI and crates.io backends, which the engines already support unchanged; a peer-to-peer transport; and formal verification of the policy engine.

Thank you. We are happy to take your questions.

[33:05] Presenter 4 · Slide 16 — References

(Do not read. Leave this slide up during questions. If asked about a source, the numbers match the literature review slides.)

Action: slide 16 (references) stays on screen for the whole Q&A.



6. If something goes wrong

What happens	Who acts	What to do
present.sh shows a red in pre-flight	Presenter 3 (before the talk)	Install the missing tool, or let it build/generate what it offers to; run demo/present.sh again.
A step says 'The scenario stopped unexpectedly'	Whoever has the keyboard	Press Enter to retry once. If it fails again, press s to skip and say: "Let me show you the same result from our recorded run", then use docs/demo-report.html.
The screen freezes or the terminal is closed by accident	Whoever has the keyboard	Open a new terminal: cd loom, then demo/present.sh --from N (N = the step you were on). It frees the ports itself.
The console page says 'Dashboard server unreachable'	Nobody (wait)	Normal between steps; it reconnects within 2 seconds of the next step starting.
Running more than 3 minutes late	Timekeeper signals; Presenter 4 decides	Skip the finale (press s at step 9) and show docs/demo-report.html for 20 seconds.
The laptop dies completely	Presenter 4	Present slides 13–14 (they contain the demo results) from any machine, and play the recorded run if you have it.

7. Questions and answers

If the question is about...	It goes to
Motivation, the AUR, related work, novelty, references	Presenter 1
Architecture, Merkle log, witnesses, signatures, hashing	Presenter 2
Sandbox (namespaces, Landlock, seccomp), policy, independence, quarantine	Presenter 3
The demo, results, evaluation, limitations, future work	Presenter 4

- Presenter 4 repeats each question in one sentence so everyone hears it, then names who will answer.
- Answer in two or three sentences, then stop. It is fine to say "that is a limitation we state in our report" when it is true.
- Closing line (Presenter 4): "Thank you for your time and your questions."